TraceDrift — Especificação de Requisitos, Metodologias e Tecnologias

Gerador sintético de traces OpenTelemetry, logs correlacionados e anomalias de observabilidade (CLI em Python)

Versão: 1.0 | **Tipo:** Ferramenta de linha de comando open source | **Saídas:** JSONL, OTLP/JSON, ingestão em OTel Collector

---

1. Visão Geral

1.1 Objetivo

O TraceDrift gera datasets sintéticos de rastreamento distribuído (spans) e logs estruturados no padrão OpenTelemetry, simulando degradações realistas de microsserviços (cascading timeouts, retry storms, saturação de filas, lock contention etc.). Serve para testar plataformas de observabilidade (Jaeger, Grafana Tempo, SigNoz, Elasticsearch) e treinar ou avaliar detectores de anomalia, sem expor dados reais (PII, tokens, IPs internos).

1.2 Escopo

Dentro do escopo:

- Leitura de fluxos de serviço (`flow.yaml`) e catálogo de anomalias.
- Geração de árvores de spans, eventos e logs correlacionados, com rótulos de verdade-terreno (*ground truth*).
- Validação estrutural e temporal.
- Exportação em JSONL interno, OTLP/JSON e envio por OTLP (HTTP/gRPC).
- Ambiente de demonstração com Collector e Jaeger ou Tempo.

Fora do escopo (v1):

- Métricas OTel (apenas traces e logs).
- Geração de carga real ou injeção de falhas em sistemas vivos.
- Interface web própria.
- Treinamento de modelos de detecção.

1.3 Personas

Persona	Necessidade
Engenheiro de SRE / Observabilidade	Popular Jaeger, Tempo ou SigNoz com traces de falha realistas
Engenheiro de ML	Dataset rotulado para detecção de anomalias e análise de causa-raiz
Desenvolvedor de ferramentas de APM	Massa de teste reproduzível e sem dados sensíveis
Instrutor / estudante	Exemplos didáticos de padrões de falha distribuída

---

2. Requisitos Funcionais (RF)

Os cinco requisitos originais foram detalhados e ampliados. Critérios no formato Dado / Quando / Então.

2.1 Entrada e configuração

RF01 — Definição de fluxo de serviço. Ler `flow.yaml` com serviços, operações, tecnologias, protocolos (HTTP, gRPC, AMQP, SQL), SLOs e latências de linha de base.

- Dado um YAML válido, Quando `tracedrift generate` é executado, Então o fluxo é validado por schema antes de qualquer chamada ao modelo.
- Dado um passo que referencia serviço inexistente, Então o programa aborta indicando campo e linha, com código de saída diferente de zero.
- Dado um fluxo com chamadas assíncronas (fila), Então a aresta é marcada como `async` e usa `SpanKind` produtor/consumidor.
- Dado um fluxo sem passos, Então o programa falha com mensagem clara.

RF02 — Catálogo de anomalias. Catálogo embutido e extensível (`anomalies.yaml`): Thundering Herd, Database Lock Contention, Circuit Breaker Tripping, Downstream Network Partition, Retry Storm, Thread Pool Exhaustion, Slow Consumer, N+1 Query, Cache Stampede, Partial DB Failure, entre outras.

- Dado o catálogo padrão, Então ele contém no mínimo 20 padrões, cada um com descrição, tecnologias aplicáveis, assinatura esperada (latência, erros, retries) e span-raiz típico da causa.

- Dado uma anomalia incompatível com o fluxo (ex.: "Lock Contention" sem banco), Então a combinação é descartada e registrada.
- Dado um `anomalies.yaml` do usuário, Então ele estende ou substitui o padrão conforme `--anomalies-mode`.

RF03 — Perfil temporal e de tráfego. Configurar janela de tempo, taxa de requisições, proporção de traces normais e anômalos e instante de início da anomalia.

- Dado `--anomaly-ratio 0.15`, Então 15% (±2 pontos) dos traces são anômalos e os demais seguem a linha de base.
- Dado `--seed 42`, Então o plano de geração é idêntico entre execuções.
- Dado `--dry-run`, Então o plano e a estimativa de tokens são exibidos sem chamar o modelo.

2.2 Geração

RF04 — Geração do blueprint do trace pela LLM. A LLM gera um *blueprint* (estrutura hierárquica com nomes, serviços, durações e deslocamentos relativos), e não IDs ou timestamps absolutos (ver seção 4).

- Dado um blueprint válido, Então cada span referencia um pai por rótulo simbólico existente e há exatamente um span-raiz.
- Dado um blueprint com ciclo ou span órfão, Então ele é rejeitado e reenviado com a descrição do erro.

RF05 — Materialização determinística (normalizer). O código converte o blueprint em spans OTel com IDs e timestamps reais.

- Dado um trace materializado, Então `trace_id` tem 32 caracteres hexadecimais não nulos e cada `span_id` tem 16, todos únicos no trace.
- Dado a mesma seed, Então os IDs gerados são reproduzíveis.
- Dado o instante-base do trace, Então `start_time_unix_nano` e `end_time_unix_nano` são inteiros em nanossegundos calculados a partir dos deslocamentos do blueprint.

RF06 — Geração de spans OTel. Cada span contém nome, serviço, kind, status, atributos e eventos.

- Dado um span com erro, Então `status.code` é `ERROR` com mensagem e existe ao menos um evento `exception` com `exception.type` e `exception.message`.
- Dado um span de banco, Então ele possui atributos `db.system` e `db.operation` (ou equivalentes das convenções semânticas vigentes).
- Dado um span HTTP, Então ele possui método, rota e código de status coerentes com o `status`.

RF07 — Logs correlacionados. Gerar registros de log ligados a spans por `trace_id` e `span_id`.

- Dado um span com erro, Então há ao menos um log `ERROR` ou `WARN` com mesmo `trace_id` e `span_id`, e timestamp dentro do intervalo do span.
- Dado cada log, Então possui `severity_text`, `severity_number` coerente, corpo e atributos de serviço.
- Dado traces normais, Então predominam logs `INFO` e a proporção de `ERROR` não excede 2%.

RF08 — Rótulos de verdade-terreno. Cada trace anômalo é acompanhado de rótulos.

- Dado um trace anômalo, Então o rótulo informa: tipo de anomalia, `root_cause_span_id`, serviços afetados, início e fim e severidade.
- Dado um trace normal, Então o rótulo é `normal`.
- Dado o relatório `root_cause_summary`, Então ele cita o serviço do `root_cause_span_id`.

RF09 — Geração em lote assíncrona com retomada.

- Dado `--concurrency 16`, Então há no máximo 16 chamadas simultâneas ao provedor.
- Dado interrupção (Ctrl+C), Então o estado é salvo e `--resume` continua sem duplicar traces.
- Dado `--max-tokens-budget`, Então a execução para limpa ao atingir o limite.

2.3 Validação e resiliência

RF10 — Validação em camadas. (1) schema do blueprint; (2) regras OTel; (3) regras temporais e semânticas; (4) regras de segurança.

- Dado falha de validação, Então a tarefa é reenviada com o erro até `--max-validation-retries` (padrão 2).
- Dado esgotadas as tentativas, Então o item vai para `rejected.jsonl` com o motivo.
- Dado o fim da execução, Então o relatório informa a taxa de aprovação por camada.

RF11 — Regras de consistência temporal.

- Dado um span filho síncrono, Então seu intervalo está contido no intervalo do pai.
- Dado um span consumidor assíncrono ligado por *link* ao produtor, Então ele pode começar após o fim do produtor, mas nunca antes do início dele.
- Dado `end_time` menor que `start_time`, Então o trace é inválido.
- Dado a soma das durações de filhos seriais, Então ela não excede a duração do pai.

- Dado o modo `--auto-fix`, Então o normalizer ajusta pequenas violações (até 5% da duração) e registra cada ajuste.

RF12 — Coerência de anomalia. A assinatura gerada deve condizir com a anomalia pedida.

- Dado "Retry Storm", Então há 3 ou mais spans de tentativa para o mesmo destino, com atributo de contagem de retry crescente.
- Dado "Cascading Timeout", Então a duração do span-raiz é maior ou igual ao timeout configurado e há erros de deadline em ao menos 2 níveis de profundidade.
- Dado "Circuit Breaker Tripping", Então aparecem spans curtos com erro de circuito aberto após uma sequência de falhas.

RF13 — Retry com backoff exponencial para HTTP 429, 5xx e timeouts da API do modelo, respeitando `Retry-After`.

- Dado erro transitório recorrente, Então o intervalo cresce exponencialmente com jitter.
- Dado erro 401/403, Então não há retry e a mensagem indica a causa.

2.4 Exportação e integração

RF14 — Exportação JSONL e OTLP/JSON.

- Dado a execução concluída, Então existem `traces.jsonl` (formato interno), `otlp_traces.json` (ou NDJSON) e `otlp_logs.json`, além de `labels.jsonl`.
- Dado o arquivo OTLP/JSON, Então ele é aceito sem erro pelo parser do protobuf do OTLP e por um OpenTelemetry Collector com o receptor adequado.
- Dado a escrita em disco, Então ela é assíncrona e não bloqueia chamadas ao modelo.

RF15 — Envio direto a coletor OTLP. Comando `tracedrift send`.

- Dado `--endpoint http://localhost:4318`, Então os traces são enviados via OTLP/HTTP, em lotes configuráveis, e a resposta de sucesso é verificada.
- Dado `--protocol grpc` e porta 4317, Então o envio usa gRPC.
- Dado `--replay-realtime`, Então os timestamps são deslocados para o instante atual preservando as durações.

RF16 — Demonstração com ferramentas de mercado. `docker-compose.yml` com OpenTelemetry Collector, Jaeger (ou Grafana Tempo + Grafana).

- Dado `docker compose up` e `tracedrift send`, Então em até 30 s os traces aparecem na UI e a árvore de spans exibida corresponde à hierarquia gerada.
- Dado o script de visualização em uma linha, Então o arquivo gerado é renderizado como árvore ou *waterfall* no terminal.

RF17 — Relatório e dataset card. `tracedrift report` e `DATASET_CARD.md`.

- Dado um dataset, Então o relatório traz: número de traces e spans, profundidade e largura médias, distribuição por anomalia e serviço, percentis de duração, taxa de erro e taxa de rejeição.
- Dado o card, Então ele declara natureza sintética, modelo, licença e limitações.

RF18 — CLI. Subcomandos `init`, `plan`, `generate`, `validate`, `send`, `report`, `visualize`.

- Dado `tracedrift --help`, Então todos aparecem com descrição.
- Dado `tracedrift validate arquivo`, Então o arquivo é checado e as linhas/traces inválidos são listados.
- Dado a chave de API, Então ela só é aceita por variável de ambiente ou `.env`.

---

3. Requisitos Não Funcionais (RNF)

ID	Categoria	Requisito	Critério de aceite mensurável
RNF01	Consistência temporal	Filhos contidos no pai (exceção: consumidores assíncronos com *link*)	100% dos traces exportados passam no validador temporal; teste de propriedade com Hypothesis
RNF02	Validação de schema	Payload validado contra a especificação OTLP antes de salvar	100% dos arquivos OTLP parseiam via protobuf (`json_format.Parse`) sem erro
RNF03	Execução assíncrona	Concorrência com `asyncio`	Com 16 workers, vazão ≥ 70% da teórica do provedor; overhead de I/O < 2%
RNF04	Baixo acoplamento	Qualquer endpoint compatível com a API da OpenAI	Testes de contrato passam em pelo menos 2 backends (ex.: vLLM/Ollama local e endpoint hospedado)
RNF05	Reprodutibilidade	Mesma seed e config geram o mesmo plano, IDs e blueprints de entrada	Hash do plano e dos prompts idêntico em 2 execuções

---

ID	Categoria	Requisito	Critério de aceite mensurável
RNF06	Escalabilidade	Datasets de até 100.000 traces	Escrita em *streaming*, memória < 1 GB, retomada funcional
RNF07	Compatibilidade OTel	Convenções semânticas atuais	Atributos validados contra tabela de convenções de uma versão fixada; versão declarada em metadados
RNF08	Realismo estatístico	Latências seguem distribuições plausíveis	Durações de linha de base seguem log-normal com p50/p99 configuráveis (desvio < 15% do alvo)
RNF09	Segurança e privacidade	Sem segredos ou PII reais	Varredura (e-mails, IPs públicos, tokens, cartões) com zero achados no dataset publicado
RNF10	Observabilidade da ferramenta	Logs estruturados e resumo de execução	Log JSON com tarefa, tentativa e latência; resumo final obrigatório
RNF11	Manutenibilidade	Código tipado e testado	Cobertura ⩾ 85% no núcleo; `mypy --strict` e `ruff` sem erros
RNF12	Portabilidade	Linux, macOS, Windows; Python 3.11+	CI em matriz 3 SO × 2 versões de Python
RNF13	Extensibilidade	Novas anomalias, exportadores e provedores por plugin	Adicionar uma anomalia exige apenas YAML e, se necessário, uma regra de validação registrada
RNF14	Licenciamento	Licenças documentadas	`LICENSE` (Apache-2.0 ou MIT), verificação de licenças no CI e aviso sobre termos do modelo

4. Modelo de Dados

4.1 Estratégia híbrida (recomendada)

Modelos de linguagem erram com frequência IDs hexadecimais, aritmética de nanossegundos e consistência temporal em árvores grandes. Por isso, a LLM não gera IDs nem timestamps absolutos:

Responsabilidade	Quem faz
Estrutura da árvore, nomes de operações, mensagens de erro, narrativa, relatório de causa-raiz	**LLM** (blueprint)
`trace_id`, `span_id`, timestamps em ns, ordenação, ajuste de coerência	**Código (normalizer)**
Validação estrutural, OTLP, regras temporais	**Código (validator)**

4.2 Blueprint (saída da LLM)

Campo	Tipo	Regra
`scenario`	string	Título do cenário, coerente com a anomalia
`anomaly_type`	enum	Pertence ao catálogo
`spans[].ref`	string	Rótulo simbólico único (ex.: `s1`)
`spans[].parent_ref`	string ou null	Exatamente um nulo (raiz); demais referenciam um `ref` existente
`spans[].service_name`	string	Existe no `flow.yaml`
`spans[].name`	string	Nome da operação
`spans[].kind`	enum	SERVER, CLIENT, PRODUCER, CONSUMER, INTERNAL
`spans[].offset_ms`	número	Deslocamento relativo ao início do pai, ≥ 0
`spans[].duration_ms`	número	> 0 e offset + duração ≤ duração do pai (exceto assíncronos)
`spans[].status`	objeto	`code` (OK, ERROR, UNSET) e `message`
`spans[].attributes`	mapa	Chaves segundo convenções semânticas

Campo	Tipo	Regra
`spans[].events[]`	lista	`name`, `offset_ms`, `attributes`
`logs[]`	lista	`span_ref`, `severity`, `body`, `offset_ms`
`root_cause_ref`	string	`ref` do span causador
`root_cause_summary`	string	Texto de diagnóstico

4.3 Saídas materializadas

- **Formato interno (`traces.jsonl`):** igual ao exemplo original, com campos em `snake_case` e `labels` embutidos.
- **OTLP/JSON:** estrutura `resourceSpans` > `scopeSpans` > `spans`, com `traceId` e `spanId` em hexadecimal, `startTimeUnixNano` e `endTimeUnixNano` como strings, atributos no formato lista de `key` + `value` tipado e `resource.attributes` com `service.name`.
- **Logs OTLP:** `resourceLogs` > `scopeLogs` > `logRecords` com `traceId`, `spanId`, `severityNumber`, `severityText` e `body`.
- **`labels.jsonl`:** `trace_id`, `anomaly_type`, `root_cause_span_id`, `affected_services`, `is_anomalous`.
- **Metadados:** `model`, `prompt_version`, `seed`, `otel_semconv_version`, `schema_version`, `synthetic: true`.

Critério de aceite: schemas do blueprint e do formato interno são exportados como JSON Schema versionado em `schema/`.

---

5. Metodologias

5.1 Processo de desenvolvimento

Prática	Aplicação
Kanban leve (ou Scrum de 1 semana)	GitHub Projects com limite de 3 itens em andamento
Schema-first	Modelos Pydantic e JSON Schema antes do gerador
TDD e testes de propriedade	Validadores e normalizer cobertos por Hypothesis (qualquer árvore válida gera traces válidos)

Prática	Aplicação
Trunk-based development	PRs pequenos, `main` sempre liberável
Conventional Commits + SemVer	Changelog e release automáticos
ADRs	Decisões em `docs/adr/` (estratégia híbrida, versão de semconv, formato OTLP)
Revisão por pares	1 aprovação e CI verde
Dogfooding de observabilidade	O próprio TraceDrift emite traces OTel da sua execução (opcional, flag `--self-trace`)

5.2 Engenharia de prompts e avaliação

- **Prompts como código:** templates Jinja2 versionados; `prompt_version` gravado nos metadados.
- **Few-shot com exemplos canônicos** por anomalia, validados pelo próprio validador.
- **Conjunto dourado:** 20 blueprints revisados manualmente para regressão.
- **Avaliação em camadas:** (1) determinística (estrutura, tempo, OTLP); (2) estatística (profundidade, distribuição de durações, diversidade); (3) LLM-as-judge opcional com modelo distinto para realismo e coerência da causa-raiz.
- **Validação por ferramentas reais:** carregar uma amostra em Jaeger ou Tempo como teste de fumaça de ingestão.
- **Revisão humana** de 5% da amostra antes da publicação.

5.3 Estratégia de testes

Nível	Ferramentas	Escopo
Unitário	pytest	Parser de fluxo, normalizer, validadores, gerador de IDs
Propriedade	Hypothesis	Invariantes: árvore única, filhos contidos, IDs únicos
Contrato	respx / mock OpenAI-compatível	Respostas 200, 429, 5xx, timeout e JSON malformado
Conformidade OTLP	opentelemetry-proto (`json_format`)	Parse de todos os arquivos exportados
Integração	Docker Compose (Collector + Jaeger)	Envio e consulta via API do Jaeger/Tempo

Nível	Ferramentas	Escopo
Ponta a ponta	CLI em CI	`init`, `plan`, `generate`, `validate`, `send`
Regressão de qualidade	Conjunto dourado + métricas	CI falha se a aprovação cair mais de 5 pontos

5.4 Definição de Pronto (DoD)

1. Critérios de aceite cobertos por testes automatizados.
2. `ruff`, `mypy` e testes passam em CI nos três sistemas operacionais.
3. Documentação atualizada (README, docstrings, `--help`).
4. Nenhum segredo ou dado pessoal adicionado (gitleaks limpo).
5. PR revisado e changelog atualizado.

Definição de Preparado (DoR): objetivo claro, critérios em Dado/Quando/Então, estimativa (P, M, G) e dependências mapeadas.

---

6. Tecnologias

6.1 Stack principal

Camada	Tecnologia	Justificativa
Linguagem	**Python 3.11+**	Ecossistema de dados, LLM e OpenTelemetry
Pacotes	**uv** (ou Poetry) e `pyproject.toml`	Instalação rápida e lockfile
CLI	**Typer** + **Rich**	Subcomandos tipados, tabelas e árvore de spans no terminal
Validação	**Pydantic v2**	Schemas, validadores customizados e JSON Schema
Cliente HTTP / LLM	**openai (AsyncOpenAI)** ou **httpx**	`base_url` configurável para qualquer provedor compatível
Assíncrono	**asyncio** + semáforos	Concorrência controlada (aiohttp é alternativa válida; httpx unifica com o cliente OTLP)
Retry	**tenacity**	Backoff exponencial com jitter

Camada	Tecnologia	Justificativa
Grafos	**networkx**	Validação de árvore (aciclicidade, raiz única, profundidade)
OTel	**opentelemetry-proto**, **opentelemetry-sdk**, **opentelemetry-exporter-otlp**	Parse/serialização OTLP oficial e envio por HTTP/gRPC
Estatística	**numpy** (e opcionalmente **scipy**)	Distribuições log-normais para latências de linha de base
Templates	**Jinja2**	Prompts versionados
Configuração	**PyYAML/ruamel.yaml**, **pydantic-settings**	`flow.yaml`, `anomalies.yaml`, variáveis de ambiente
I/O	**aiofiles** (ou fila + thread de escrita)	Escrita assíncrona em streaming
Logs da ferramenta	**structlog**	Logs JSON

6.2 Modelos e provedores (compatíveis com a API da OpenAI)

Opção	Uso
Nemotron (NVIDIA NIM ou hospedado)	Geração principal
vLLM	Auto-hospedagem com *guided decoding* por JSON schema
Ollama / llama.cpp server	Desenvolvimento e testes locais
Groq e similares	Alta velocidade para volumes grandes

Interface `LLMClient` com `base_url`, `model` e `api_key`; saída estruturada preferencialmente por `response_format` /JSON schema, com fallback de parsing tolerante. Confirmar a licença do modelo antes de publicar o dataset.

6.3 Qualidade, CI/CD e distribuição

Finalidade	Ferramenta
Lint, formatação	**ruff**

Finalidade	Ferramenta
Tipagem	**mypy** (estrito)
Testes	**pytest**, **pytest-asyncio**, **pytest-cov**, **Hypothesis**, **respx**
Hooks	**pre-commit**
CI	**GitHub Actions** (3 SO × Python 3.11 e 3.12; job de integração com `services` Docker)
Segurança	**gitleaks**, **pip-audit**, **Dependabot**, **CodeQL**
Release	**release-please** + **PyPI** (Trusted Publishing)
Contêiner	**Docker** multi-stage
Documentação	**MkDocs Material**

6.4 Plataforma de observabilidade para demonstração

Componente	Papel
OpenTelemetry Collector (distribuição contrib)	Receber OTLP (4317 gRPC, 4318 HTTP) e encaminhar
Jaeger (all-in-one)	UI de traces para demo rápida
Grafana Tempo + Grafana	Alternativa com busca TraceQL e correlação com logs
Grafana Loki (opcional)	Armazenar e correlacionar logs
SigNoz (opcional)	Plataforma completa de referência
telemetrygen (do Collector contrib)	Comparação: gerador de carga trivial vs. cenários ricos do TraceDrift

---

7. Arquitetura

```
[flow.yaml] + [anomalies.yaml] + [perfil temporal]
        |
        ▼
   Planner (seed, proporção normal/anômalo, filtros de compatibilidade)
        |
        ▼
```

```
   Prompt Composer (Jinja2, prompt_version)
        |
        ▼
   LLMClient (async, semáforo, retry, orçamento de tokens)
        |
        ▼
   Blueprint Validator (schema + grafo: raiz única, sem ciclos)
        |
        ▼
   Normalizer (IDs, timestamps em ns, ajuste de coerência)
        |
        ▼
   Validador OTel/Temporal/Segurança  ── inválido ──▶ retry corretivo ──▶
rejected.jsonl
        |
        ▼
   Exporters: JSONL interno | OTLP/JSON | envio OTLP (HTTP/gRPC) | labels
        |
        ▼
   Relatório e dataset card
```

7.1 Módulos

Módulo	Responsabilidade
`models.py`	Pydantic: fluxo, anomalia, blueprint, span, log, label
`flow.py`	Carga do YAML e grafo de serviços
`planner.py`	Plano determinístico de geração
`prompts/`	Templates e exemplos few-shot
`client.py`	Interface `LLMClient` e cliente compatível com OpenAI
`generator.py`	Orquestração assíncrona, retomada, orçamento
`normalizer.py`	Materialização de IDs e tempos, auto-ajuste
`validator.py`	Regras OTel, temporais, de anomalia e segurança
`exporter.py`	JSONL, OTLP/JSON e envio ao Collector
`report.py`	Estatísticas e dataset card
`viz.py`	Visualização em terminal (árvore e waterfall)
`cli.py`	Comandos Typer

---

8. Segurança e Conformidade

- **Segredos:** chave de API apenas por variável de ambiente; mascarada em logs.
- **Dados sensíveis:** valores de atributos (user agents, IPs, e-mails) vindos de pools fictícios; IPs em faixas reservadas (RFC 5737 e RFC 1918). Critério: nenhum IP público real no dataset.
- **Injeção de prompt via YAML:** textos livres do fluxo são tratados como dados, delimitados e limitados em tamanho. Critério: uma descrição maliciosa não altera o formato da saída.
- **Rotulagem:** `synthetic: true` nos metadados e dataset card.
- **Envio a coletores:** `send` exige confirmação explícita (`--yes`) quando o endpoint não é `localhost` , para evitar poluir ambientes de produção.
- **Marcação nos recursos:** `resource.attributes` inclui `deployment.environment=synthetic` e `tracedrift.generated=true` , permitindo filtrar facilmente.
- **Cadeia de suprimentos:** dependências travadas, `pip-audit` e Dependabot.

9. Métricas de Sucesso

Métrica	Meta
Aprovação do blueprint na 1ª tentativa	⩾ 90%
Aprovação final (com retries)	⩾ 98%
Traces OTLP aceitos por Collector, Jaeger/Tempo	100%
Traces com violação temporal após normalização	0%
Coerência de anomalia (verificação RF12)	⩾ 95%
Diversidade (profundidade, largura e durações)	Distribuição com desvio-padrão relativo ⩾ 25%
Nota LLM-as-judge (1 a 5)	⩾ 4,0
Throughput	Meta de 10.000 traces/hora com modelo local e 16 workers (medir e publicar)
Cobertura de testes no núcleo	⩾ 85%
Tempo de instalação até ver o primeiro trace no Jaeger	< 10 minutos

10. Riscos e Mitigações

Risco	Impacto	Mitigação
LLM erra IDs, tempos e hierarquia em árvores grandes	Alto	Estratégia híbrida: a LLM só gera o blueprint; código materializa IDs e tempos
Contexto muito longo para traces com dezenas de spans	Médio	Limitar spans por chamada, gerar subárvores e compor, ou gerar templates reutilizáveis variados por código
Padrões repetitivos entre traces	Alto	Perfis estatísticos, seeds distintas e atributos sorteados por código

Risco	Impacto	Mitigação
Divergência das convenções semânticas do OTel (atributos renomeados)	Médio	Fixar e declarar a versão de semconv; tabela de mapeamento
Dados incompatíveis com a ferramenta de destino	Médio	Testes de integração com Collector, Jaeger e Tempo no CI
Rate limit ou cota insuficiente	Médio	Backoff, orçamento de tokens, retomada
Restrições de licença do modelo sobre dados gerados	Alto	Revisar termos antes de publicar e registrar no card

---

11. Roadmap

Marco	Entregas	Duração estimada
M0 — Fundação	Repositório, CI, modelos Pydantic, fluxo e catálogo de anomalias	1 semana
M1 — Núcleo	Planner, cliente, blueprint, normalizer, validador temporal	2 semanas
M2 — OTel e logs	OTLP/JSON, logs correlacionados, labels, validação por protobuf	1 semana
M3 — Integração	`send`, Docker Compose com Collector e Jaeger/Tempo, visualização	1 semana
M4 — Qualidade e escala	Concorrência, retomada, relatório, conjunto dourado, LLM-as-judge	1 semana
M5 — Publicação	Dataset de exemplo, dataset card, documentação, PyPI, v1.0.0	1 semana

---

12. Backlog Inicial (épicos)

Épico	Histórias principais
E1 — Configuração	Parser de `flow.yaml`; catálogo de anomalias; camadas de configuração
E2 — Planejamento	Perfil temporal; proporção normal/anômalo; `--dry-run` com estimativa de tokens
E3 — Geração	Prompt composer; cliente assíncrono; blueprint; retry; retomada
E4 — Materialização	Normalizer; IDs; timestamps; spans assíncronos e *links*
E5 — Validação	Regras OTel; temporais; coerência de anomalia; segurança
E6 — Logs e rótulos	Logs correlacionados; `labels.jsonl`; relatório de causa-raiz
E7 — Exportação e integração	JSONL; OTLP/JSON; `send`; Docker Compose; visualização
E8 — Qualidade e distribuição	Relatório; dataset card; CI/CD; PyPI; Hugging Face

---

13. Pontos de Atenção no Documento Original

1. **O exemplo não é OTLP/JSON.** O formato mostrado usa `snake_case` e atributos como mapa simples. O OTLP/JSON usa `camelCase`, `resourceSpans` / `scopeSpans`, atributos como lista tipada e timestamps como strings. Defina o formato interno e implemente um conversor para OTLP.
2. **IDs e timestamps gerados pela LLM.** São fonte frequente de erros; recomenda-se a estratégia híbrida (seção 4).
3. **RNF01 precisa de exceção.** Em mensageria assíncrona, o consumidor pode terminar depois do produtor (relacionados por *link*); a regra "filho sempre dentro do pai" deve valer para chamadas síncronas.
4. **Convenções semânticas desatualizadas.** No exemplo, `http.status_code` e `http.method` foram substituídos nas convenções estáveis por `http.response.status_code` e `http.request.method`, e `user_agent` por `user_agent.original`. Fixe a versão de semconv e valide as chaves.

5. **Status.** Em OTel, os códigos são `UNSET`, `OK` e `ERROR`; no OTLP/JSON o enum aparece como `STATUS_CODE_ERROR` ou número. A mensagem só é relevante quando o código é `ERROR`.
6. **Escopo de logs.** O RF04 pede logs correlacionados, mas o formato de saída só mostra spans; defina o formato de logs e a ligação por `trace_id` e `span_id`.
7. **Rótulos para ML.** O objetivo de treinar detectores de anomalia exige rótulos explícitos (verdade-terreno), ausentes na proposta.
8. **Uso de tokens.** Árvores com dezenas de spans podem gerar saídas muito longas; avalie gerar o blueprint compacto (sem atributos repetitivos) e deixar o código expandir atributos padrão por tipo de span.
9. **RNF04 "Execução gratuita".** Reformular como "sem dependência de um provedor específico".

---

14. Estrutura de Repositório Revisada

```
tracedrift/
├── .github/workflows/ci.yml, release.yml
├── configs/ (e_commerce_flow.yaml, fintech_flow.yaml,
anomalies_default.yaml)
├── datasets/ (sample_traces_100.jsonl, sample_otlp.json,
sample_labels.jsonl)
├── deploy/ (docker-compose.yml, otel-collector-config.yaml, tempo.yaml)
├── docs/ (adr/, guia, dataset card)
├── schema/ (blueprint.schema.json, trace.schema.json)
├── tracedrift/
|   ├── __init__.py, cli.py, models.py, flow.py, planner.py
|   ├── client.py, generator.py, normalizer.py, validator.py
|   ├── exporter.py, report.py, viz.py
|   └── prompts/ (blueprint_v1.j2, correction_v1.j2, judge_v1.j2)
├── tests/ (unit, property, contract, integration, golden)
├── pyproject.toml, README.md, LICENSE, CHANGELOG.md
└── Dockerfile
```